# Schema-Validated Event Ingestion with Backpressure-Aware Routing

A throughput study of event ingestion under JSON Schema enforcement and downstream-consumer fan-out

**Manikanta Reddy Mandadhi**

Senior Data Scientist — Independent Research

2026

# Contents

# 1. Abstract

Event-driven architectures are the dominant pattern for decoupled, scalable data pipelines, but production deployments regularly fail two ways: schema drift breaks downstream consumers, and slow consumers cause back-pressure cascades that block producers. We design an event ingestion API with JSON-Schema validation at the boundary, per-consumer queues, and explicit back-pressure handling. We benchmark on a synthetic event-stream load of 100,000 events/sec across 12 schema types with three downstream consumers of varying processing speed. Validation throughput is 14,800 events/sec/core; at 6 cores, the API ingests over 80,000 events/sec sustained. Slow-consumer back-pressure is contained per-consumer without affecting others. Schema-evolution scenarios (additive, field-rename, breaking) are evaluated with documented rollout patterns.

**Keywords:** event-driven architecture, JSON Schema, back-pressure, stream processing, schema evolution

# 2. Introduction

Event pipelines are hard to operate well: most published designs assume well-formed input and well-behaved consumers. Production deployments meet neither assumption. The research problem is to design an ingestion API that catches schema violations at the boundary (rather than at the consumer), handles slow-consumer back-pressure without producer impact, and supports schema evolution gracefully.

## 2.1 Research Problem

Event pipelines are hard to operate well: most published designs assume well-formed input and well-behaved consumers. Production deployments meet neither assumption. The research problem is to design an ingestion API that catches schema violations at the boundary (rather than at the consumer), handles slow-consumer back-pressure without producer impact, and supports schema evolution gracefully.

## 2.2 Research Questions and Hypotheses

**Research question:** Can JSON-Schema validation throughput exceed 10,000 events/sec/core?

*Hypothesis:* We expect 12-18k events/sec/core based on the published Ajv benchmark.

**Research question:** Does per-consumer queue isolation prevent slow-consumer back-pressure cascades?

*Hypothesis:* We expect zero impact on fast consumers when slow consumers are saturated.

**Research question:** Do additive schema changes deploy without consumer-side change?

*Hypothesis:* We expect full backward compatibility with appropriately structured additive changes.

**Research question:** Can the API ingest at the design throughput (100k events/sec) with 8-core hardware?

*Hypothesis:* We expect feasibility based on the per-core throughput estimate.

# 3. Literature Review

## 3.1 Theories Grounding the Problem

1. **Schema Evolution Theory (Roddick, 1995)** — Schema changes classify as additive, retractive, or impactive; only impactive changes require coordinated client updates, motivating field-defaulting and version-aware deserialization. (Roddick (1995))
2. **Back-Pressure (Akka Streams team, 2014)** — Reactive Streams' Subscriber.request(n) protocol formalises back-pressure as a first-class concern; per-consumer demand limits prevent cascading saturation. (Akka Streams (2014))
3. **JSON Schema (IETF Draft)** — JSON Schema provides a portable, declarative validation language; Ajv's compilation strategy provides the throughput needed for boundary enforcement. (Wright et al. (2020))
4. **Log-Structured Event Storage (Kreps, 2014)** — Storing events in an append-only log with per-consumer offsets enables independent consumer progress and replay; this is the structural basis for the per-consumer queue design. (Kreps (2014))
5. **Bulkhead Pattern (Hohpe & Woolf, 2003)** — Per-consumer queues isolate slow-consumer impact from fast consumers; the same bulkhead pattern as in microservice resilience. (Hohpe & Woolf (2003))

## 3.2 Supporting Examples

- Apache Kafka's per-partition consumer offset is the production-scale analogue of the per-consumer queue design.
- Confluent Schema Registry implements the schema evolution patterns formalized here at platform scale.
- AWS EventBridge and Google Pub/Sub provide managed-service equivalents whose internal designs inform this work.

# 4. Research Method

The API is a Node.js service using Ajv for JSON-Schema validation. Validated events route to per-consumer Redis Streams. Each consumer has an independent worker pool and back-pressure policy. We evaluate validation throughput, ingestion throughput at varying core counts, slow-consumer isolation, and three schema-evolution scenarios. Synthetic load is generated with k6.

# 5. Data Description

**Source:** Synthetic event stream with 12 schema types — Generated by simulator scripts in this repository

**Coverage:** 100k events/sec sustained × 30 min = 180 million events; 3 consumer simulators

**Schema (selected fields):**

- event_id, schema_id, schema_version, payload
- ingest_ts, validation_status
- per-consumer offsets and throughput

**Preprocessing:** Schema diversity calibrated to a representative SaaS event catalog (page_view, signup, purchase, …). Slow-consumer saturation simulated by injected processing delay.

**License / availability:** Synthetic.

# 6. Analysis

## 6.1 Validation throughput

Per-core JSON Schema validation throughput as a function of payload size.

| Payload class | Mean payload size | Events/sec/core |
|---|---|---|
| Small (page_view) | 0.5 KB | 21,400 |
| Medium (signup) | 2 KB | 14,800 |
| Large (purchase) | 8 KB | 6,200 |

## 6.2 Ingestion throughput by core count

Sustained ingestion throughput on the medium-payload mix.

| Cores | Sustained throughput | p95 latency |
|---|---|---|
| 1 | 14.4k eps | 12 ms |
| 2 | 27.1k eps | 14 ms |
| 4 | 52.8k eps | 17 ms |
| 6 | 80.4k eps | 21 ms |
| 8 | 104.2k eps | 24 ms |

### 6.3 Slow-consumer isolation

Throughput on fast consumers when one slow consumer is saturated.

| Consumer | Steady throughput | During slow-consumer saturation |
|---|---|---|
| Fast (low-latency) | 47k eps | 47k eps (no change) |
| Medium | 21k eps | 21k eps (no change) |
| Slow (saturated) | 6k eps | 6k eps (capped) |

### 6.4 Schema evolution outcomes

Behavior under three change classes with the canonical rollout pattern.

| Change class | Producer impact | Consumer impact | Rollout window |
|---|---|---|---|
| Additive (new optional) | Deploy first | None (default to ignore) | Same hour |
| Field rename | Dual-write old+new | Read both during transition | 1-2 weeks |
| Breaking | New version namespace | Coordinated upgrade | Coordinated |

## 7. Discussion

All four hypotheses are supported. Validation throughput is well above the 10k events/sec/core target on small and medium payloads; large payloads hit the expected slowdown. The API ingests over 100k events/sec on 8 cores. Slow-consumer back-pressure is contained perfectly per-consumer. The three schema-evolution scenarios all execute cleanly with the documented rollout patterns. The most operationally consequential lesson is that boundary validation pays for itself the first time it catches a malformed producer: the alternative — propagating malformed events to consumers — produces incident debugging that costs orders of magnitude more than the validation overhead.

## 8. Conclusion

An ingestion API with JSON-Schema validation, per-consumer back-pressure isolation, and explicit schema-evolution patterns delivers production-grade reliability at six-figure events/sec throughput. The artefact and its rollout patterns are released as a reference.

## 9. Future Work

- Move from JSON Schema to Avro plus a schema registry for binary efficiency.

- Add a dead-letter queue with replay for validation rejections.
- Layer in event-time-windowed consumer aggregations.
- Cross-region replication for global ingestion.

## 10. References

1. Kreps, J. (2014). *I □ Logs: Event Data, Stream Processing, and Data Integration.* O'Reilly.
2. JSON Schema Specification. https://json-schema.org/specification.html
3. Roddick, J. F. (1995). *A Survey of Schema Versioning Issues for Database Systems.* Information and Software Technology 37(7).
4. Wright, A. et al. (2020). *JSON Schema: A Media Type for Describing JSON Documents.* IETF Draft. https://json-schema.org/specification.html